Chapter 19
Data Binding and the Form Tag Library

Data binding is a feature that binds user input to the domain model. Thanks to data binding, HTTP request parameters, which are always of type **String**, can be used to populate object properties of various types. Data binding also makes form beans (e.g. instances of **ProductForm** in the previous chapters) redundant.

To use data binding effectively, you need the Spring form tag library. This chapter explains data binding and the form tag library and provides examples that highlight the use of the tags in the form tag library.

Data Binding Overview

Due to the nature of HTTP, all HTTP request parameters are strings. Recall that in the previous chapters you had to parse a string to a float in order to get the correct product price. To refresh your memory, here is some code from the **ProductController** class's **saveProduct** method in **app18a**.

```
@RequestMapping(value="product_save")
public String saveProduct(ProductForm productForm,
        Model model) {
    logger.info("saveProduct called");
    // no need to create and instantiate a ProductForm
    // create Product
    Product product = new Product();
    product.setName(productForm.getName());
    product.setDescription(productForm.getDescription());
    try {
        product.setPrice(Float.parseFloat(
            productForm.getPrice()));
    } catch (NumberFormatException e) {
    }
```

You had to parse the **price** property in the **ProductForm** because it was a String and you needed a float to populate the **Product**'s **price** property. With data binding you can replace the **saveProduct** method fragment above with this.

```
@RequestMapping(value="product_save")
public String saveProduct(Product product, Model model)
```

Thanks to data binding, you don't need the **ProductForm** class anymore and no parsing is necessary for the price property of the Product object.

Another benefit of data binding is for repopulating an HTML form when input validation fails. With manual HTML coding, you have to worry about repopulating the input fields with the values the user previously entered. With Spring data binding and the form tag library, this is taken care of for you.

The Form Tag Library

The form tag library contains tags you can use to render HTML elements in your JSP pages. To use the tags, declare this **taglib** directive at the top of your JSP pages.

```
<%@taglib prefix="form"
    uri="http://www.springframework.org/tags/form" %>
```

Table 19.1 shows the tags in the form tag library.

Each of the tags will be explained in the following subsections. A sample application presented in the section, "Data Binding Example" demonstrates the use of data binding with the form tag library.

Tag	Description
form	Renders a form element.
input	Renders an ____ element
password	Renders an element
hidden	Renders an <input type="hidden"/> element
textarea	Renders a textarea element
checkbox	Renders an ☐ element
checkboxes	Renders multiple ☐ elements
radiobutton	Renders an ☐ element
radiobuttons	Renders multiple ☐ elements
select	Renders a select element
option	Renders an option element.
options	Renders a list of option elements.
errors	Renders field errors in a span element.

Table 19.1: The Form tags

The form Tag

The **form** tag renders an HTML form. You must have a **form** tag to use any of the other tags that render a form input field. The attributes of the **form** tag are given in Table 19.2.

All attributes in Table 19.2 are optional. The table does not include HTML attributes, such as **method** and **action**.

Attribute	Description
acceptCharset	Specifies the list of character encodings accepted by the server.
commandName	The name of the model attribute under which the form object is exposed. The default is 'command.'
cssClass	Specifies the CSS class to be applied to the rendered form element.
cssStyle	Specifies the CSS style to be applied to the rendered form element
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
modelAttribute	The name of the model attribute under which the form-backing object is exposed. The default is 'command'.

Table 19.2: The form tag's attributes

The **commandName** attribute is probably the most important attribute as it specifies the name of the model attribute that contains a backing object whose properties will be used to populate the generated form. If this attribute is present, you must add the corresponding model attribute in the request-handling method that returns the view containing this form. For instance, in the **app19a** application that accompanies this chapter, the following **form** tag is specified in the **BookAddForm.jsp**.

```
<form:form commandName="book" action="book_save" method="post">
    ...
</form:form>
```

The **inputBook** method in the **BookController** class is the request-handling method that returns **BookAddForm.jsp**. Here is the **inputBook** method.

```
@RequestMapping(value = "/book_input")
public String inputBook(Model model) {
    ...
    model.addAttribute("book", new Book());
    return "BookAddForm";
}
```

As you can see, a **Book** object is created and added to the **Model** with attribute name **book**. Without the model attribute, the **BookAddForm.jsp** page will throw an exception because the **form** tag cannot find a form-backing object specified in its **commandName** attribute.

In addition, you will still normally use the **action** and **method** attributes. Both are HTML attributes and therefore not included in Table 19.2.

The input Tag

The **input** tag renders an **<input type="text"/>** element. The most important attribute of this tag, the **path** attribute, binds this input field to a property of the form-backing object. For example, if the **commandName** attribute of the enclosing **<form/>** tag is assigned **book** and the **path** attribute of the **input** tag is given the value **isbn**, the **input** tag will be bound to the **isbn** property of the **Book** object.

Table 19.3 shows all the attributes in the **input** tag. All attributes in Table 19.3 are optional and the table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
CssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
path	The path to the property to bind.

Table 19.3: The input tag's attributes

As an example, this **input** tag is bound to the **isbn** property of the form-backing object.

```
<form:input id="isbn" path="isbn" cssErrorClass="errorBox"/>
```

This will be rendered as the following **<input/>** element:

```
<input type="text" id="isbn" name="isbn"/>
```

The **cssErrorClass** attribute has no effect unless there is an input validation error in the **isbn** property and the same form is used to redisplay the user input, in which case the **input** tag will be rendered as this **input** element.

```
<input type="text" id="isbn" name="isbn" class="errorBox"/>
```

The **input** tag can also be bound to a property in a nested object. For example, the following **input** tag is bound to the **id** property of the **category** property of the form-backing object.

```
<form:input path="category.id"/>
```

The password Tag

The **password** tag renders an **<input type="password"/>** element and its attributes are given in Table 19.4. The **password** tag is similar to the **input** tag except that it has a **showPassword** attribute.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
path	The path to the property to bind.
showPassword	Indicates whether the password should be shown rather than masked. The default is false.

Table 19.4: The password tag's attributes

All attributes in Table 19.4 are optional and the table does not include HTML attributes. Here is an example of the **password** tag.

```
<form:password id="pwd" path="password" cssClass="normal"/>
```

The hidden Tag

The **hidden** tag renders an **<input type="hidden"/>** element and its attributes are given in Table 19.5. The **hidden** tag is similar to the **input** tag except that it has no visual appearance and therefore does not support a **cssClass** or **cssStyle** attribute.

Attribute	Description
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
path	The path to the property to bind.

Table 19.5: The hidden tag's attributes

All attributes in Table 19.5 are optional and the table does not include HTML attributes.

The following is an example **hidden** tag.

```
<form:hidden path="productId"/>
```

The textarea Tag

The **textarea** tag renders an HTML **textarea** element. As you know, a **textarea** element is basically an **input** element that supports multiline input. The attributes of the textarea tag are presented in Table 19.6. All attributes in Table 19.6 are optional and the table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
path	The path to the property to bind.

Table 19.6: The password tag's attributes

For example, the following **textarea** tag is bound to the **note** property of the form-backing object.

```
<form:textarea path="note" tabindex="4" rows="5" cols="80"/>
```

The checkbox Tag

The **checkbox** tag renders an **☐** element. The attributes that can appear within the **checkbox** tag are listed in Table 19.7. All attributes are optional and this table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
label	The value to be used as the label for the rendered checkbox.
path	The path to the property to bind.

Table 19.7: The checkbox tag's attributes

For example, the following **checkbox** tag is bound to the **outOfStock** property.

```
<form:checkbox path="outOfStock" value="Out of Stock"/>
```

The radiobutton Tag

The **radiobutton** tag renders an **☐** element. The attributes for **radiobutton** are given in Table 19.8. All attributes in Table 19.8 are optional and the table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
label	The value to be used as the label for the rendered radio button.
path	The path to the property to bind.

Table 19.8: The radiobutton tag's attributes

For instance, the following **radiobutton** tags are bound to a **newsletter** property.

```
Computing Now <form:radiobutton path="newsletter"
        value="Computing Now"/> <br/>
Modern Health <form:radiobutton path="newsletter"
        value="Modern Health"/>
```

The checkboxes Tag

The **checkboxes** tag renders multiple **☐** elements. The attributes that may appear within **checkboxes** are given in Table 19.9. All attributes are optional and the table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
delimiter	Specifies a delimiter between two input elements. By default, there is no delimiter.
element	Specifies an HTML element to enclosed each rendered input element. The default is 'span'.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
items	The Collection, Map, or array of objects used to generate the input elements.
itemLabel	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the label for each input element.
itemValue	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the value for each input element.
path	The path to the property to bind.

Table 19.9: The checkboxes tag's attributes

For example, the following **checkboxes** tag renders the content of model attribute **categoryList** as check boxes. The **checkboxes** tag allows multiple selections.

```
<form:checkboxes path="category" items="${categoryList}"/>
```

The radiobuttons Tag

The **radiobuttons** tag renders multiple **☐** elements. The attributes for **radiobuttons** tag are presented in Table 19.10.

For example, the following **radiobuttons** tag renders the content of model attribute **categoryList** as radio buttons. Only one radio button can be selected at a time.

<form:radiobuttons path="category" items="${categoryList}"/>

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
delimiter	Specifies a delimiter between two input elements. By default, there is no delimiter.
element	Specifies an HTML element to enclosed each rendered input element. The default is 'span'.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
items	The Collection, Map, or array of objects used to generate the input elements.
itemLabel	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the label for each input element.
itemValue	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the value for each input element.
path	The path to the property to bind.

Table 19.10: The radiobuttons tag's attributes

The select Tag

The **select** tag renders a HTML select element. The options for the rendered element may come from a collection or a map or an array assigned to its **items** attribute or from a nested **option** or **options** tag. The properties of the **select** tag are given in Table 19.11. All attributes are optional and none of the HTML attributes is included in the table.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
items	The Collection, Map, or array of objects used to generate the input elements.
itemLabel	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the label for each input element.
itemValue	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the value for each input element.
path	The path to the property to bind.

Table 19.11: The select tag's attributes

The **items** attribute is particularly useful as it may be bound to a collection, a map, or an array of objects to generate the options for the **select** element.

For example, the following **select** tag is bound to the **id** property of the **category** property of the form-backing object. Its options come from a **categories** model attribute. The value for each option comes from the **id** property of the objects in the **categories** collection/map/array, and its label comes from the **name** property.

```
<form:select id="category" path="category.id"
    items="${categories}" itemLabel="name"
    itemValue="id"/>
```

The option Tag

The **option** tag renders an HTML **option** element to be used within a **select** element. Its attributes are given in Table 19.12. All attributes are optional and the table does not include HTML attributes.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.

Table 19.12: The option tag's attributes

For example, here is an example **option** tag.

```
<form:select id="category" path="category.id"
        items="${categories}" itemLabel="name"
        itemValue="id">
    <option value="0">-- Please select --</option>
</form:select>
```

This code snippet renders a **select** element whose options come from a **categories** model attribute as well as from the **option** tag.

The options Tag

The **options** tag generates a list of HTML option elements. Table 19.13 shows the attributes that may appear in the options tag. It does not include HTML attributes.

The **app19a** application provides an example of the **options** tag.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
cssErrorClass	Specifies the CSS class to be applied to the rendered input element if the bound property contains errors, overriding the value of the cssClass attribute.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
items	The Collection, Map, or array of objects used to generate the input elements.
itemLabel	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the label for each input element.
itemValue	The property of the objects in the collection/Map/array specified in the items attribute that is to supply the value for each input element.

Table 19.13: The options tag's attributes

The errors Tag

The **errors** tag renders one or more HTML **span** element that each contains a field error message. This tag can be used to display a specific field error or all field errors.

The attributes of the **errors** tag are listed in Table 19.14. All attributes are optional and the table does not include HTML attributes that may appear in the HTML **span** elements.

Attribute	Description
cssClass	Specifies the CSS class to be applied to the rendered input element.
cssStyle	Specifies the CSS style to be applied to the rendered input element
delimiter	Delimiter for separating multiple error messages.
element	Specifies an HTML element to enclose the error messages.
htmlEscape	Accepts true or false indicating whether or not the rendered value(s) should be HTML-escaped.
path	The path to the errors object to bind.

Table 19.14: The errors tag's attributes

For example, this **errors** tag displays all field errors.

```
<form:errors path="*"/>
```

The following **errors** tag displays a field error associated with the **author** property of the form-backing object.

```
<form:errors path="author"/>
```

Data Binding Example

As an example of using the tags in the form tag library to take advantage of data binding, consider the **app19a** application. This example centers around the **Book** domain class. The class has several properties, including a **category** property of type **Category**. **Category** has two properties, **id** and **name**.

The application allows you to list books, add a new book, and edit a book.

The Directory Structure

Figure 19.1 shows the directory structure of **app19a**.

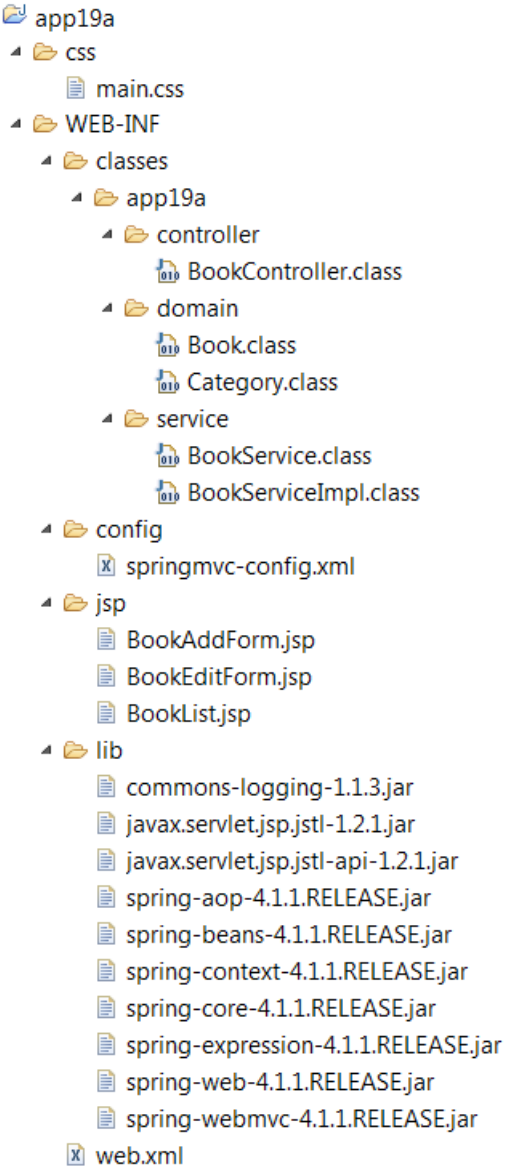

Figure 19.1: The directory structure of app19a

The Domain Classes

The **Book** class and the **Category** class are the domain classes in this application. They are given in Listing 19.1 and Listing 19.2, respectively.

Listing 19.1: The Book class

```
package app19a.domain;

import java.io.Serializable;

public class Book implements Serializable {

    private static final long serialVersionUID =
            1520961851058396786L;
    private long id;
    private String isbn;
    private String title;
    private Category category;
    private String author;

    public Book() {
    }

    public Book(long id, String isbn, String title,
            Category category, String author) {
        this.id = id;
        this.isbn = isbn;
        this.title = title;
        this.category = category;
        this.author = author;
    }

    // get and set methods not shown

}
```

Listing 19.2: The Category class

```
package app19a.domain;

import java.io.Serializable;

public class Category implements Serializable {
    private static final long serialVersionUID =
            565871679395790 4104L;
    private int id;
    private String name;

    public Category() {
    }

    public Category(int id, String name) {
        this.id = id;
```

```
        this.name = name;
    }

    // get and set methods not shown

}
```

The Controller Class

The example provides a controller for **Book**, the **BookController** class. It allows the user to create a new book, update a book's details, and list all books in the system. Listing 19.3 shows the **BookController** class.

Listing 19.2: The BookController class

```
package app19a.controller;

import java.util.List;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import app19a.domain.Book;
import app19a.domain.Category;
import app19a.service.BookService;

@Controller
public class BookController {

    @Autowired
    private BookService bookService;

    private static final Log logger =
       LogFactory.getLog(BookController.class);

    @RequestMapping(value = "/book_input")
    public String inputBook(Model model) {
        List<Category> categories = bookService.getAllCategories();
        model.addAttribute("categories", categories);
        model.addAttribute("book", new Book());
        return "BookAddForm";
    }

    @RequestMapping(value = "/book_edit/{id}")
    public String editBook(Model model, @PathVariable long id) {
        List<Category> categories = bookService.getAllCategories();
        model.addAttribute("categories", categories);
        Book book = bookService.get(id);
        model.addAttribute("book", book);
```

```
        return "BookEditForm";
    }

    @RequestMapping(value = "/book_save")
    public String saveBook(@ModelAttribute Book book) {
        Category category =
       bookService.getCategory(book.getCategory().getId());
        book.setCategory(category);
        bookService.save(book);
        return "redirect:/book_list";
    }

    @RequestMapping(value = "/book_update")
    public String updateBook(@ModelAttribute Book book) {
        Category category =
       bookService.getCategory(book.getCategory().getId());
        book.setCategory(category);
        bookService.update(book);
        return "redirect:/book_list";
    }

    @RequestMapping(value = "/book_list")
    public String listBooks(Model model) {
        logger.info("book_list");
        List<Book> books = bookService.getAllBooks();
        model.addAttribute("books", books);
        return "BookList";
    }
}
```

BookController is dependent on a **BookService** for some back-end processing. The **@Autowired** annotation is used to inject an instance of **BookService** implementation to the **BookController**.

```
@Autowired
private BookService bookService;
```

The Service Class

Finally, Listing 19.4 and Listing 19.5 show the **BookService** interface and the **BookServiceImpl** class, respectively. As the name implies, **BookServiceImpl** implements **BookService**.

Listing 19.4: The BookService interface

```
package app19a.service;

import java.util.List;
import app19a.domain.Book;
import app19a.domain.Category;

public interface BookService {
```

```
    List<Category> getAllCategories();
    Category getCategory(int id);
    List<Book> getAllBooks();
    Book save(Book book);
    Book update(Book book);
    Book get(long id);
    long getNextId();

}
```

Listing 19.5: The BookServiceImpl class

```
package app19a.service;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;
import app19a.domain.Book;
import app19a.domain.Category;

@Service
public class BookServiceImpl implements BookService {

    /*
     * this implementation is not thread-safe
     */
    private List<Category> categories;
    private List<Book> books;

    public BookServiceImpl() {
        categories = new ArrayList<Category>();
        Category category1 = new Category(1, "Computing");
        Category category2 = new Category(2, "Travel");
        Category category3 = new Category(3, "Health");
        categories.add(category1);
        categories.add(category2);
        categories.add(category3);

        books = new ArrayList<Book>();
        books.add(new Book(1L, "9780980839623",
                "Servlet & JSP: A Tutorial",
                category1, "Budi Kurniawan"));
        books.add(new Book(2L, "9780980839630",
                "C#: A Beginner's Tutorial",
                category1, "Jayden Ky"));
    }

    @Override
    public List<Category> getAllCategories() {
        return categories;
    }

    @Override
```

```
public Category getCategory(int id) {
    for (Category category : categories) {
        if (id == category.getId()) {
            return category;
        }
    }
    return null;
}

@Override
public List<Book> getAllBooks() {
    return books;
}

@Override
public Book save(Book book) {
    book.setId(getNextId());
    books.add(book);
    return book;
}

@Override
public Book get(long id) {
    for (Book book : books) {
        if (id == book.getId()) {
            return book;
        }
    }
    return null;
}

@Override
public Book update(Book book) {
    int bookCount = books.size();
    for (int i = 0; i < bookCount; i++) {
        Book savedBook = books.get(i);
        if (savedBook.getId() == book.getId()) {
            books.set(i, book);
            return book;
        }
    }
    return book;
}

@Override
public long getNextId() {
    // needs to be locked
    long id = 0L;
    for (Book book : books) {
        long bookId = book.getId();
        if (bookId > id) {
            id = bookId;
        }
```

```
        }
        return id + 1;
    }
}
```

The **BookServiceImpl** class contains a **List** of **Book** objects and a **List** of **Category** object. Both lists are populated when the class is instantiated. The class also contains methods for retrieving all books, retrieve a single book, and add and update a book.

The Configuration File

Listing 19.6 presents the Spring MVC configuration file for **app19a**.

Listing 19.6: The Spring MVC configuration file

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/mvc
        http://www.springframework.org/schema/mvc/spring-mvc.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-
       context.xsd">

    <context:component-scan base-package="app19a.controller"/>
    <context:component-scan base-package="app19a.service"/>

    ... <!-- other elements are not shown -->

</beans>
```

The **component-scan** beans causes the **app19a.controller** and **app19a.service** packages to be scanned.

The View

The three JSP pages used in **app19a** are given in Listings 19.7, 19.8, and 19.9. In the **BookAddForm.jsp** and **BookEditForm.jsp** pages, the tags from the form tag library are used.

Listing 19.7: The BookList.jsp page

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<title>Book List</title>
```

```
<style type="text/css">@import url("<c:url
       value="/css/main.css"/>");</style>
</head>
<body>

<div id="global">
<h1>Book List</h1>
<a href="<c:url value="/book_input"/>">Add Book</a>
<table>
<tr>
    <th>Category</th>
    <th>Title</th>
    <th>ISBN</th>
    <th>Author</th>
    <th> </th>
</tr>
<c:forEach items="${books}" var="book">
    <tr>
        <td>${book.category.name}</td>
        <td>${book.title}</td>
        <td>${book.isbn}</td>
        <td>${book.author}</td>
        <td><a href="book_edit/${book.id}">Edit</a></td>
    </tr>
</c:forEach>
</table>
</div>
</body>
</html>
```

Listing 19.8: The BookAddForm.jsp page

```
<%@ taglib prefix="form"
        uri="http://www.springframework.org/tags/form" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE HTML>
<html>
<head>
<title>Add Book Form</title>
<style type="text/css">@import url("<c:url
       value="/css/main.css"/>");</style>
</head>
<body>

<div id="global">
<form:form commandName="book" action="book_save" method="post">
    <fieldset>
        <legend>Add a book</legend>
        <p>
            <label for="category">Category: </label>
             <form:select id="category" path="category.id"
                items="${categories}" itemLabel="name"
                itemValue="id"/>
```

```
        </p>
        <p>
            <label for="title">Title: </label>
            <form:input id="title" path="title"/>
        </p>
        <p>
            <label for="author">Author: </label>
            <form:input id="author" path="author"/>
        </p>
        <p>
            <label for="isbn">ISBN: </label>
            <form:input id="isbn" path="isbn"/>
        </p>

        <p id="buttons">
            <input id="reset" type="reset" tabindex="4">
            <input id="submit" type="submit" tabindex="5"
                value="Add Book">
        </p>
    </fieldset>
</form:form>
</div>
</body>
</html>
```

Listing 19.9: The BookEditForm.jsp page

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/form"
      %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<title>Edit Book Form</title>
<style type="text/css">@import url("<c:url
      value="/css/main.css"/>");</style>
</head>
<body>

<div id="global">
<form:form commandName="book" action="/book_update" method="post">
    <fieldset>
        <legend>Edit a book</legend>
        <form:hidden path="id"/>
        <p>
            <label for="category">Category: </label>
             <form:select id="category" path="category.id" items="$
      {categories}"
                itemLabel="name" itemValue="id"/>
        </p>
        <p>
            <label for="title">Title: </label>
            <form:input id="title" path="title"/>
```

```
            </p>
            <p>
                <label for="author">Author: </label>
                <form:input id="author" path="author"/>
            </p>
            <p>
                <label for="isbn">ISBN: </label>
                <form:input id="isbn" path="isbn"/>
            </p>

            <p id="buttons">
                <input id="reset" type="reset" tabindex="4">
                <input id="submit" type="submit" tabindex="5"
                    value="Update Book">
            </p>
        </fieldset>
</form:form>
</div>
</body>
</html>
```

Testing the Application

To test the application, go to this page.

```
http://localhost:8080/app19a/book_list
```

Figure 19.2 shows the list of books when the application is first started.

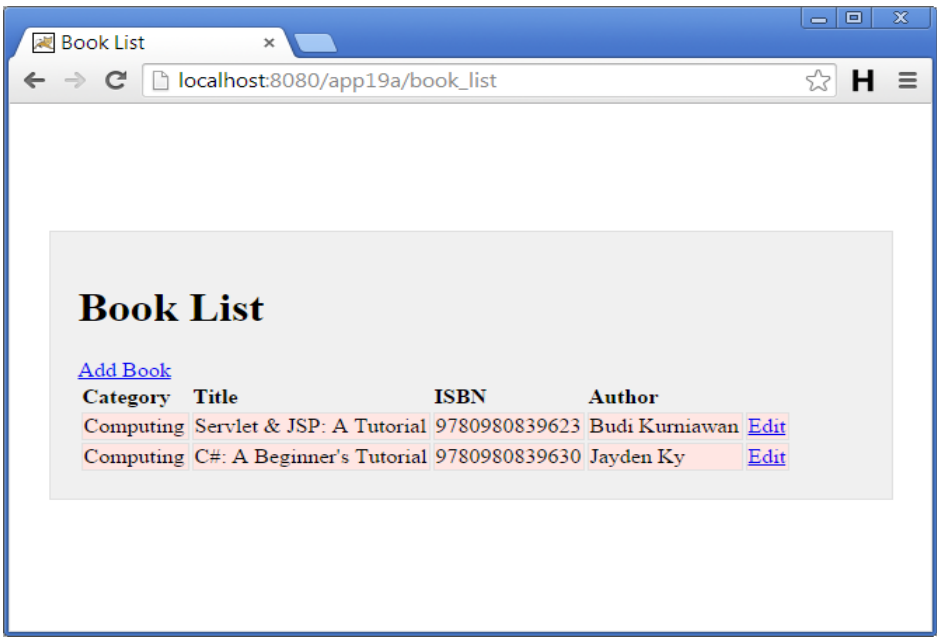

Figure 19.2: The book list

You can click the Add Book link to add a book or an Edit link to the right of a book's details to edit the book.

Figure 19.3 shows the Add Book form and Figure 19.4 the Edit Book form.

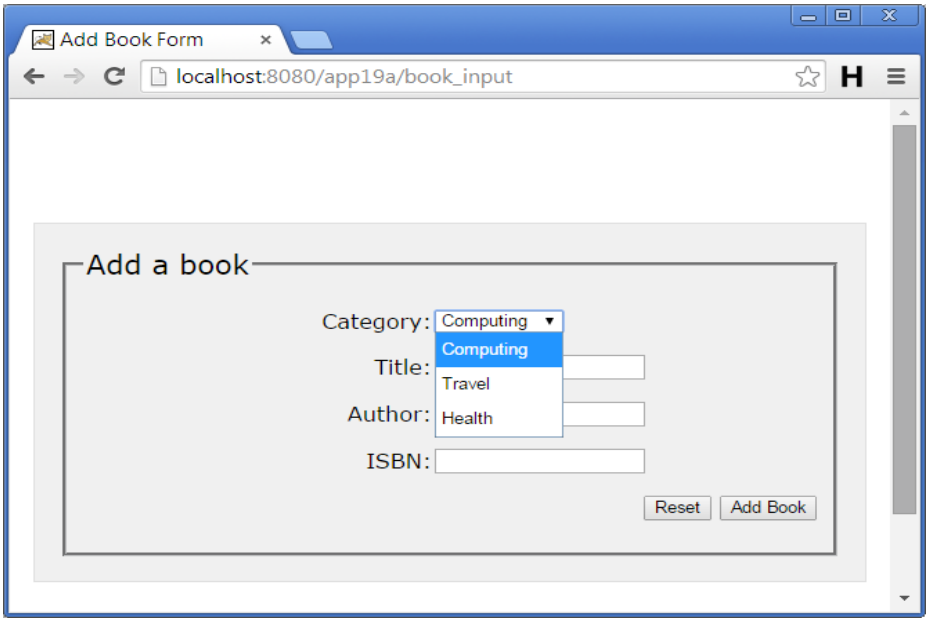

Figure 19.3: The Add Book form

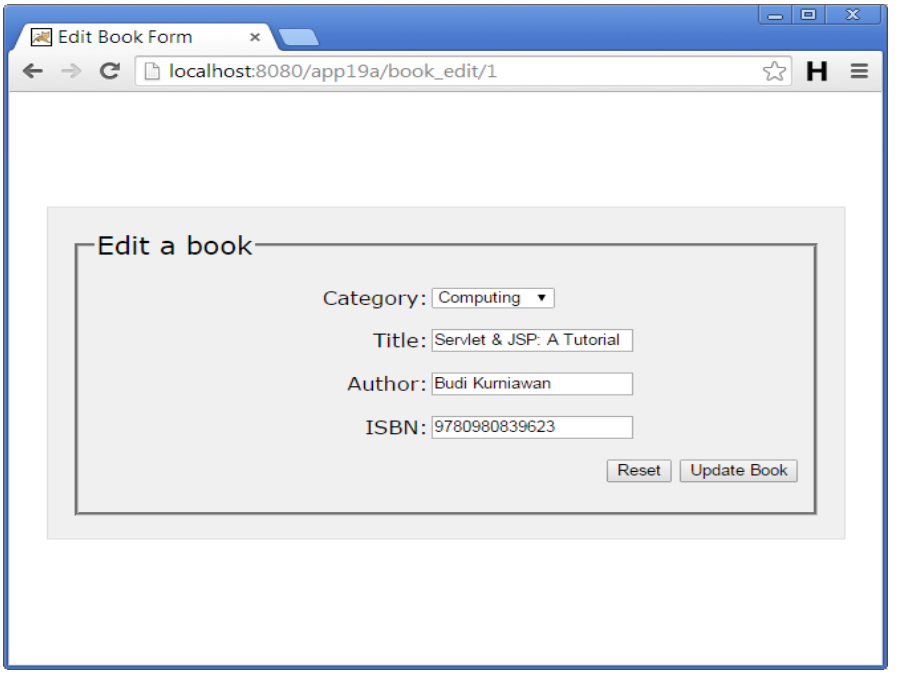

Figure 19.4: The Edit Book form

Summary

In this chapter you learned about data binding and the tags in the form tag library. The next two chapters discuss how you can further take advantage of data binding with converters, formatters, and validators.